1 LLM APP STACK

2 RAG BOOK

3 AGENT LOOP

4 MULTI-AGENT SWARM

5 GATEWAY TOLLBOOTH

6 ROUTER SIGNPOST

7 VECTOR CONSTELLATION

8 DATA CONVEYOR

LIGHTNING STREAM

10 BATCH CALENDAR

11 HUMAN CHECKPOINT

12 MULTI-MODEL PUZZLE

13 FAULT-TOLERANT SHIELD

14 SCALING LADDER

15 SECURITY PADLOCK

AI SYSTEM DESIGN

The Complete Handbook — 15 Architectures Powering Real AI Products.

Design
Build
Ship
Impact

Architect for Scale. Design for Real World.

TECH FUSIONIST

Scan to Connect

WHAT'S INSIDE

- [ ] LLM Application Architecture
- [ ] RAG Architecture
- [ ] Agent Architecture
- [ ] Multi-Agent Architecture
- [ ] AI Gateway Architecture
- [ ] Model Router Architecture

- [ ] Vector Search Architecture
- [ ] AI Data Pipeline
- [ ] Real-time AI Pipeline
- [ ] Batch AI Pipeline

- [ ] Human-in-the-loop Architecture
- [ ] Multi-model Architecture
- [ ] Fault-tolerant AI Architecture
- [ ] Scalable AI Architecture
- [ ] Secure AI Architecture

TECH FUSIONIST

Scan to Connect

LLM APPLICATION ARCHITECTURE

The skeleton under every chatbot, copilot and AI feature you've shipped.

WHY THIS EXISTS

Calling a model API straight from application code feels fast for a demo and collapses in production. Prompts get buried across the codebase so nobody can version or A/B them. There's no memory between turns, no token budget, no schema on the way out, and no record of what a request cost. The moment a second developer touches it, or the model provider changes a response format, everything silently breaks.

CLIENT LAYER
web / mobile / IDE plugin / Slack bot

streams tokens, handles retry UX, never holds an API key

API + AUTH LAYER
your backend, session management, tenant isolation

one user's context must never leak into another's prompt

ORCHESTRATION LAYER
prompt assembly, tool binding, context window budgeting

the brain of the app — versioned prompts live here, not in code

CONTEXT LAYER
system prompt + conversation memory + retrieved docs + user profile

everything competing for a fixed token budget

MODEL LAYER
provider SDK call, streaming, temperature & max-token config

swappable on purpose — never hard-code one vendor

POST-PROCESS LAYER
schema validation, output filter, citation attach, cost logging

an LLM returns text, not guarantees — validate before you trust

DESIGN DECISIONS

- Stateless backend + external memory store — so any server can handle any turn
- Prompt templates as versioned config artifacts, never inline strings
- Every model call wrapped: timeout, retry-with-backoff, token count, trace ID

NEW HERE?

Context window = the model's short-term memory, measured in tokens, and it's a hard ceiling. Token ≈ ¾ of a word. Orchestration layer = the code that decides what goes into the prompt before the model ever sees it.

TECH FUSIONIST

Scan to Connect

RAG ARCHITECTURE

How a bot answers from YOUR documents instead of guessing from training data.

WHY THIS EXISTS

A model only knows what it was trained on, so it will answer confidently about your refund policy, your architecture docs or last quarter's numbers and be entirely wrong. Fine-tuning on your corpus is slow, costly and stale the moment a document changes, and it still gives you no way to show where an answer came from. RAG sidesteps all of it by fetching the relevant text at query time and making the model read before it speaks.

THE LEVERS

- Chunks too small → context is fragmented and meaning is lost
- Chunks too large → irrelevant text crowds out the real answer
- Top-K too high → noise; too low → the right passage never arrives
- No metadata filter → users retrieve documents they shouldn't be allowed to see

INDEXING — runs offline, on a schedule

SOURCE DOCS → PARSE & CLEAN → CHUNK → EMBED → UPSERT with metadata → VECTOR DATABASE (embeddings + metadata)

- SOURCE DOCS: PDFs, Confluence, tickets, code
- PARSE & CLEAN: strip headers, OCR scans
- CHUNK: 400-800 tokens, 10-15% overlap, split on headings not characters
- EMBED: [0.1 0.2 ...] text → 768/1536-dim vector
- UPSERT with metadata: source, date, ACL, version

chunk size is the single biggest quality lever in RAG

RETRIEVAL — runs per user query, in milliseconds

USER QUERY → QUERY REWRITE → EMBED QUERY → HYBRID SEARCH → RERANK → PROMPT ASSEMBLY → LLM → CITED ANSWER

- QUERY REWRITE: resolve "it"/"that one" using chat history
- EMBED QUERY: [0.1 0.2...]
- HYBRID SEARCH: vector similarity + BM25 keyword, fused
- RERANK: cross-encoder trims top-50 down to top-5
- PROMPT ASSEMBLY: system prompt + retrieved chunks + question

retrieval finds candidates; reranking decides truth

NEW HERE?

- [0.1 0.2...] Embedding = text converted into a list of numbers so that similar meanings land near each other.
- Chunk = a small slice of a document.
- Top-K = how many chunks you retrieve.

RAG does not teach the model anything — it just hands it an open book.

TECH FUSIONIST

Scan to Connect

AGENT ARCHITECTURE

The difference between an AI that talks about booking a flight and one that actually books it.

WHY THIS EXISTS

A plain model call is a single shot — it produces text and stops. It cannot check a live price, read a file, run a query, or correct itself after seeing a result. Anything that takes several dependent steps, where step three depends on what step two returned, cannot be done in one prompt. The loop is what turns a text generator into something that can pursue a goal, and the guards are what stop it pursuing that goal forever.

AGENT STATE

GOAL
the immutable original objective

SCRATCHPAD
running reasoning trace

TOOL RESULTS
observations so far

MEMORY
facts carried across sessions

→ every loop iteration re-sends this — context grows, cost grows, quality degrades. Summarise or prune.

THINK
model reasons about the goal and picks the next action

ACT
emits a structured tool call (function name + typed arguments)

- Web search
- Code execution
- Database query
- File I/O
- Internal API
- Calendar/ email

OBSERVE
tool result comes back, gets appended to working context

EVALUATE
done? blocked? retry? escalate? → loop again or exit

TERMINATION GUARD
max steps, max cost, max wall-clock, goal-satisfied check

WHERE AGENTS FAIL

- **Infinite loops** — the model retries the same failing call with the same arguments
- **Context bloat** — by step 15 the scratchpad crowds out the original goal
- **Silent wrong-tool calls** — no schema validation on arguments before execution

NEW HERE?

Tool = a function you expose to the model with a name, a description and typed arguments.

The model doesn't run it — it asks your code to run it and waits for the answer.

An agent is just a model stuck in a think-act-observe loop with tools taped to its hands.

TECH FUSIONIST

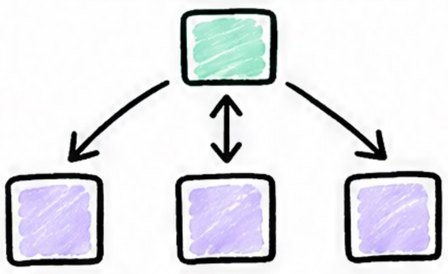
Scan to Connect

MULTI-AGENT ARCHITECTURE

Division of labour, not more AI —
a newsroom instead of one exhausted writer.

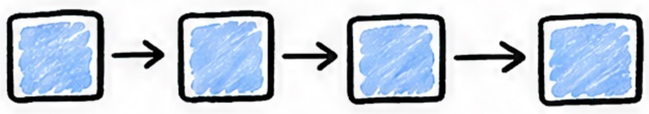
PICK YOUR TOPOLOGY

SUPERVISOR



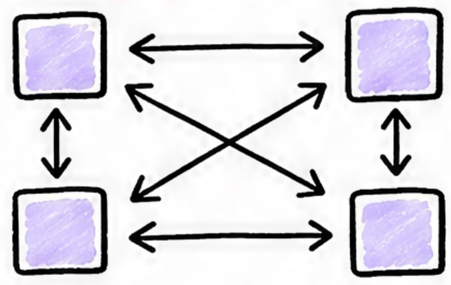
predictable, debuggable,
the orchestrator is the bottleneck

PIPELINE



great for fixed workflows,
brittle if a stage fails mid-chain

PEER NETWORK



flexible, expensive,
hardest to trace when it goes wrong

WORKED EXAMPLE: RESEARCH REPORT

ORCHESTRATOR
decomposes goal, assigns tasks, merges results.

RESEARCHER
TOOLS: web search, doc retrieval
gather sources, no opinions

ANALYST
TOOLS: code execution, data query
compute, chart, quantify

CRITIC
TOOLS: retrieval only
verify claims against sources, flag gaps

SHARED CONTEXT
shared scratchpad — the handoff medium

SYNTHESIZER
single merged answer

WHY THIS EXISTS

One agent given a sprawling objective degrades badly — its context fills with unrelated intermediate work, its system prompt has to describe every possible behaviour at once, and there is no independent check on its output because the thing that wrote the answer is also the thing grading it. Splitting into narrow roles keeps each context small and focused, and lets you put a genuinely separate reviewer between the work and the user.

THE REAL COSTS

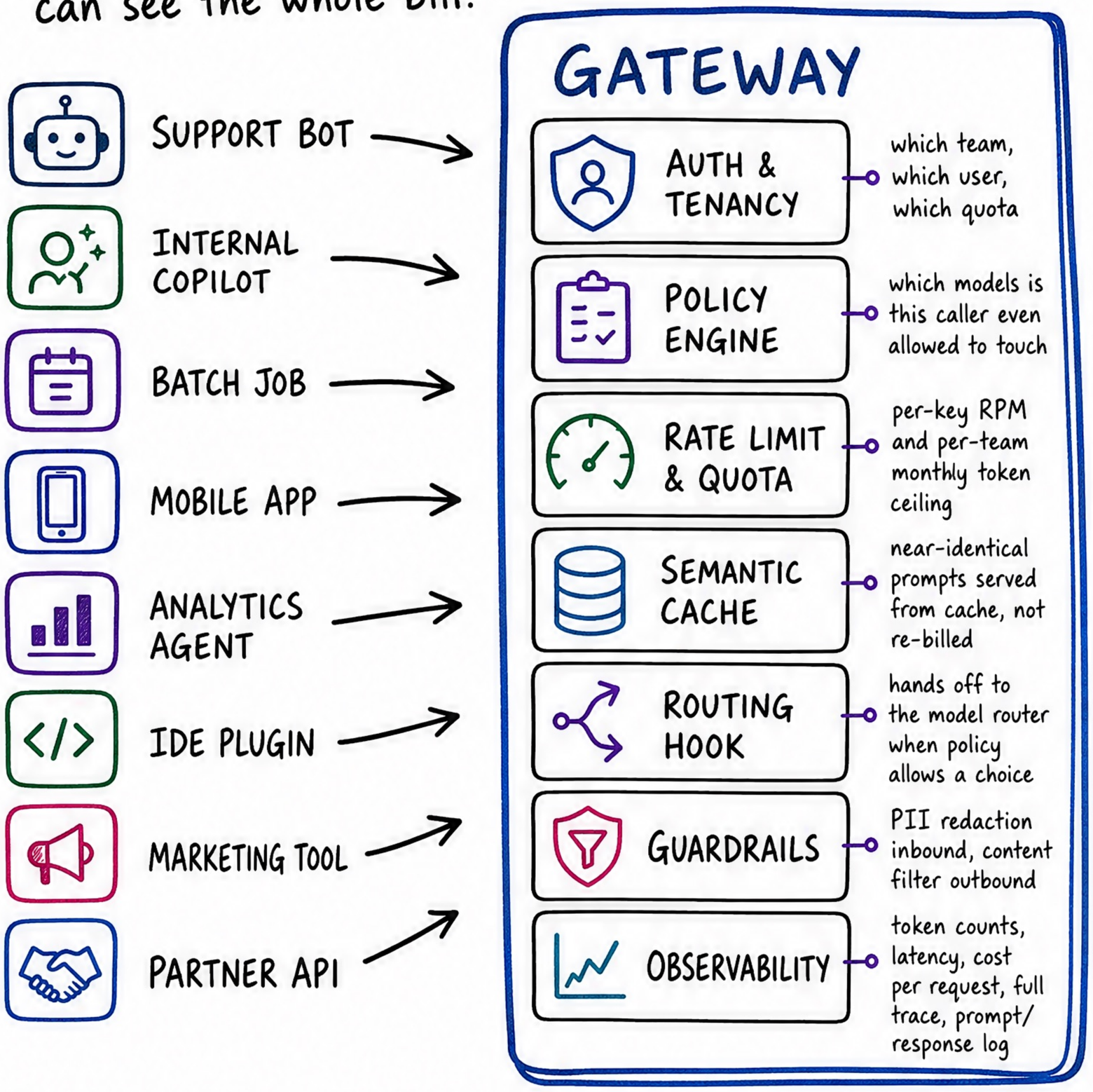
- Token spend multiplies — four agents can mean four full contexts per task
- Latency compounds — sequential handoffs add up fast
- Debugging is harder — you now trace a conversation, not a call stack

NEW HERE?

Orchestrator = the agent that only plans and delegates, never does the work.
Handoff = one agent passing task plus context to another.
Start with supervisor topology — reach for peer networks only when you've proven you need them.

AI GATEWAY ARCHITECTURE

One door every AI request walks through — and the only place you can see the whole bill.



WHY THIS EXISTS

Once more than one team is calling model APIs directly, nobody can answer basic questions: what did we spend last month, which team caused the spike, who has access to which model, what prompts were sent, and are we leaking customer data into a third-party log. Keys get copied into notebooks and scripts and never rotated, one runaway retry loop can burn a month's budget overnight, and switching provider means editing a dozen repos. Centralising the egress path is what makes all of that a config change instead of a migration.

WHAT A GATEWAY LETS YOU FINALLY MEASURE

- Cost per team, per feature, per customer
- Cache hit rate — often 20–40% on repetitive internal workloads
- P95 latency split by provider
- Blocked requests: policy violations vs PII hits vs quota exhaustion

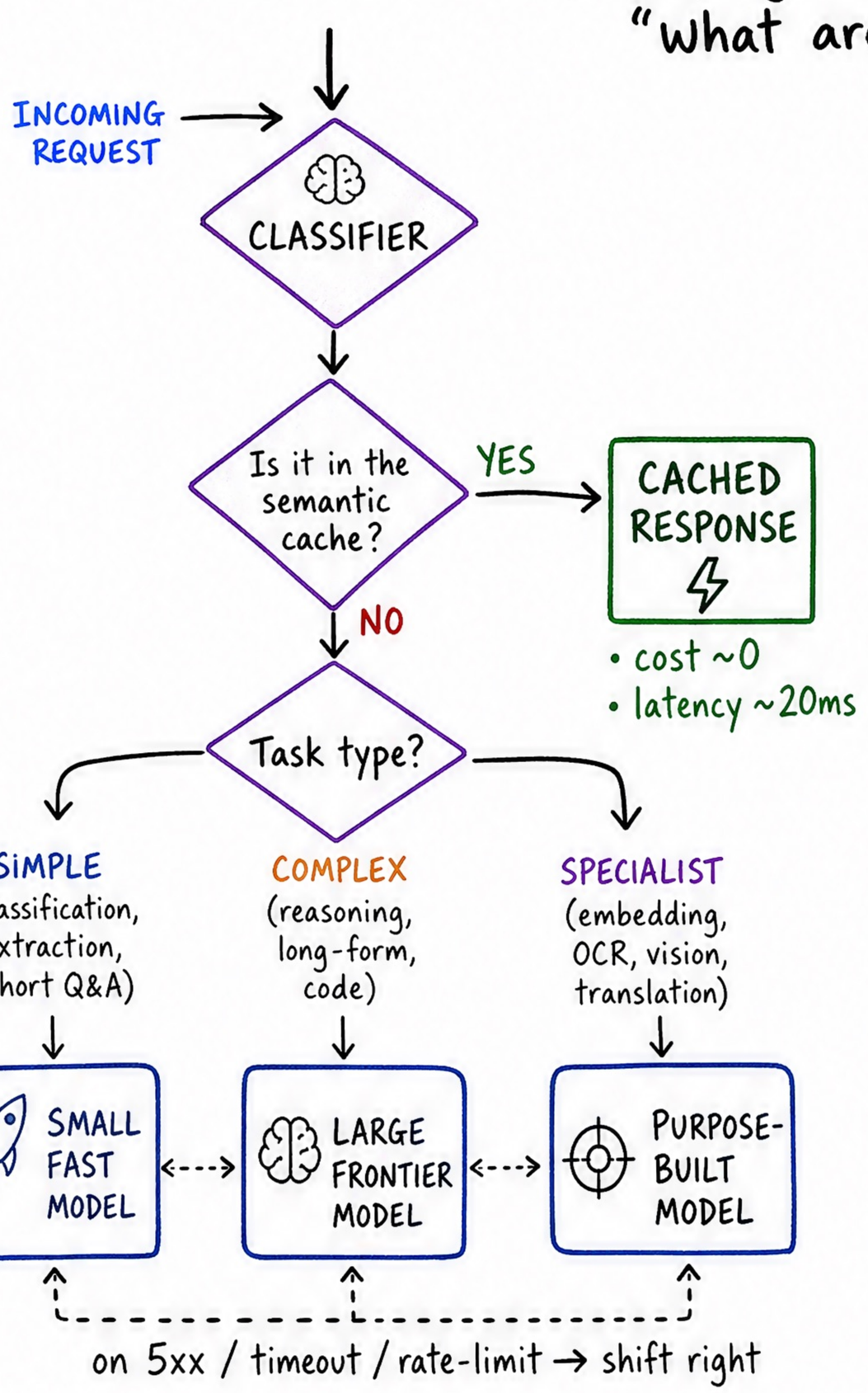
NEW HERE?

A gateway is a reverse proxy that happens to understand AI traffic.

- Semantic cache = cache that matches on meaning, not on an exact string.
- Egress = traffic leaving your systems toward an external provider.

MODEL ROUTER ARCHITECTURE

Stop paying frontier-model prices to answer "what are your opening hours'.



ROUTING RULES

Signal	Route to	Why	Watch out for
Prompt token count under 200, no tools	SMALL MODEL	most traffic, most savings	classifier itself costs tokens — keep it tiny or heuristic
Contains code block or schema	FRONTIER MODEL	correctness matters more than cost	don't route code to a small model to save pennies
User is on a paid tier	FRONTIER MODEL	quality is the product	tier-aware routing
Provider P95 latency breaching SLO	SECONDARY PROVIDER	protects user experience	needs live health signals, not static config
Confidence of small model below threshold	ESCALATE TO FRONTIER	cascade pattern	you pay twice on escalation — measure the escalation rate

WHY THIS EXISTS

Most production AI traffic is not hard. Classification, short answers, formatting and extraction make up the bulk of calls, and sending all of it to the largest available model means paying an order of magnitude more than the work is worth while also inheriting that model's latency. Meanwhile, a single provider becomes a single point of failure — when it degrades, everything degrades together. Routing turns model choice into a runtime decision instead of a hard-coded one.

TWO WAYS TO ROUTE

1. Predictive routing — classify up front, send once. Cheaper, risks misclassification.
2. Cascade routing — try small first, escalate on low confidence. Safer, pays twice sometimes.

NEW HERE?

SLO = the latency or reliability target you've promised.
Cascade = try cheap, fall back to expensive.
The router is a dispatcher: easy questions go to the intern, hard ones to the expert, and nobody waits on the wrong desk.

TECH FUSIONIST

Scan to Connect

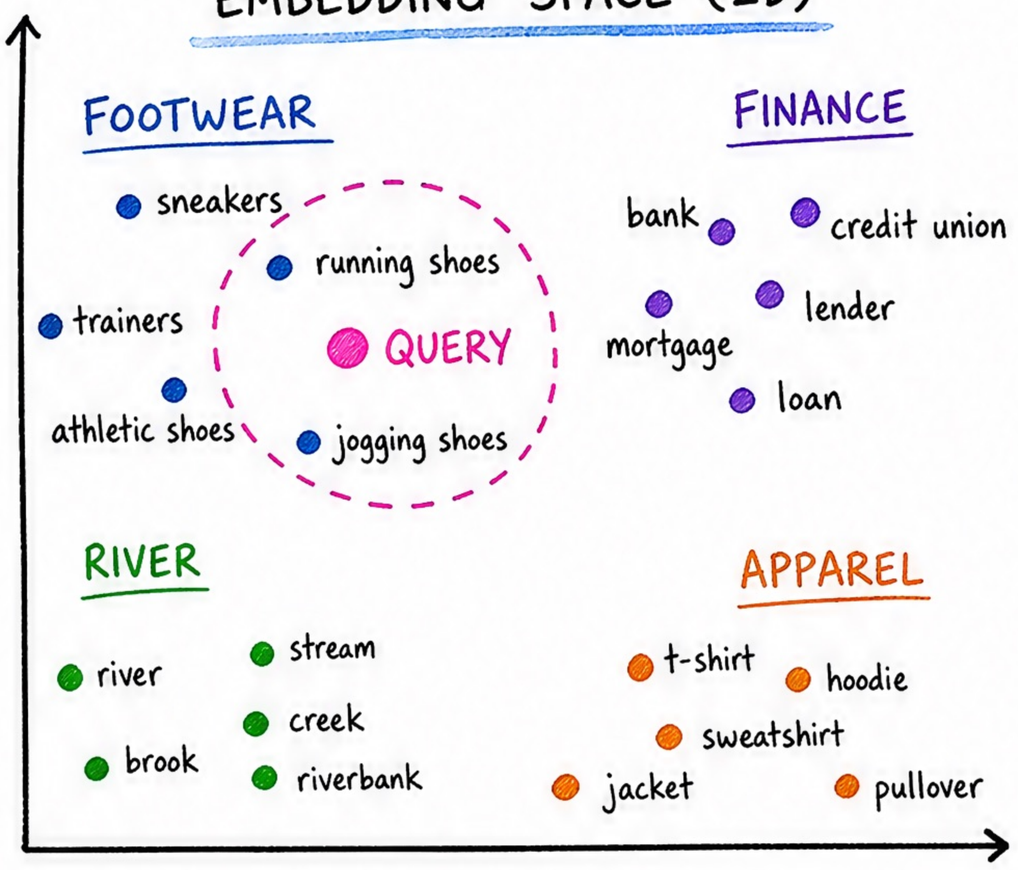
VECTOR SEARCH ARCHITECTURE

Search that knows 'sneakers' and 'running shoes' are the same thing — and 'bank' isn't always money.

WHY THIS EXISTS

Keyword search matches characters, so it misses synonyms, paraphrases and intent, and it has no concept of a document being 'about' something. Comparing meaning directly across millions of documents at query time is computationally impossible if done naively — checking every vector against every query does not survive contact with production traffic. Vector indexes make the search approximate on purpose, trading a sliver of recall for orders of magnitude in speed.

EMBEDDING SPACE (2D)



distance = dissimilarity — cosine or dot product, not keyword overlap

real vectors live in 768–3072 dimensions; this is a flattened cartoon of that

bad embeddings = bad clusters = bad search, and no amount of tuning fixes it downstream

NEW HERE?

- ANN = approximate nearest neighbour — deliberately imperfect search that's fast enough to be useful.
- Recall = the share of genuinely relevant results you actually got back.
- Vector search finds things that MEAN the same, not things SPELLED the same.

HOW THE INDEX ACTUALLY WORKS

INDEX	HOW IT SEARCHES	TRADE-OFF
FLAT / brute force	compares the query to every vector	perfect recall, unusable past a few hundred thousand vectors
HNSW	walks a layered proximity graph, hopping closer each layer	fast and accurate, memory-hungry, slow to build
IVF	pre-clusters vectors, searches only the nearest clusters	cheap and compact, recall drops if you probe too few clusters
PQ / quantisation	compresses each vector into a small code	huge memory savings, measurable accuracy loss — usually layered on top of IVF or HNSW

THE FOUR KNOBS

- Dimensions — bigger isn't better, it's just more expensive
- ef_search / nprobe — the recall vs latency dial
- Metadata filters — must be applied at index time, not after retrieval
- Re-embedding — change the embedding model and the entire index is invalid

TECH FUSIONIST

Scan to Connect

DATA PIPE

AI DATA PIPELINE

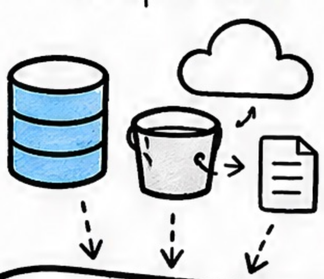
The unglamorous plumbing that decides whether your model is brilliant or embarrassing.

WHY THIS EXISTS

Every AI failure that looks like a model problem is usually a data problem wearing a disguise. Sources disagree on formats and IDs, duplicates inflate retrieval results, timezone and encoding drift corrupts comparisons, and documents arrive with no record of where they came from or who is allowed to see them. Without a pipeline that validates, normalises and stamps provenance on every record, you cannot reproduce a result, explain an answer, or safely delete a customer's data when they ask.

1 INGEST

Postgres CDC, S3 drops, SaaS APIs, event stream, scraped docs



Pull vs push · schema registry at the boundary · record source + timestamp

2 VALIDATE

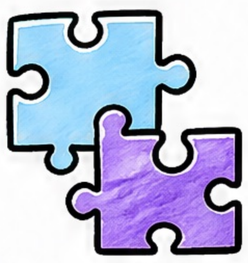
reject before you propagate

Schema checks, null thresholds, range checks, dedupe on content hash

QUALITY GATE – bad records park in a dead-letter queue, they never silently pass.

3 TRANSFORM

normalise into one shape



Unify encodings and timezones, flatten nesting, resolve entity IDs

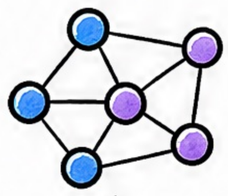
4 ENRICH / LABEL

add what the model needs

PII tagging, ACL stamping, language detect, human or weak labels

5 EMBED / FEATURIZE

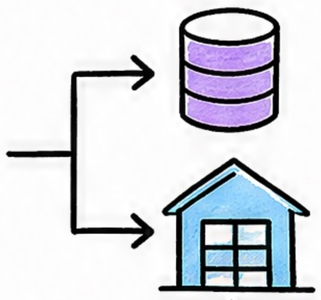
turn text and events into model input



Batch the embed calls · version the embedding model in metadata

6 LOAD

two destinations, on purpose



Vector store for retrieval · warehouse/feature store for training + analytics

NON-NEGOTIABLES

- ☑ **Idempotency** — rerunning the pipeline must not duplicate anything
- ☑ **Lineage** — every stored chunk traces back to a source document and version
- ☑ **Backfill path** — you WILL need to reprocess history after a bug or a model change
- ☑ **Deletion path** — GDPR-style removal must propagate to the vector store too

NEW HERE?

- **CDC** = change data capture, streaming row changes out of a database.
- **Dead-letter queue** = where bad records go to be inspected instead of silently dropped.
- **Lineage** = knowing exactly where a piece of data came from.

TECH FUSIONIST

Scan to Connect

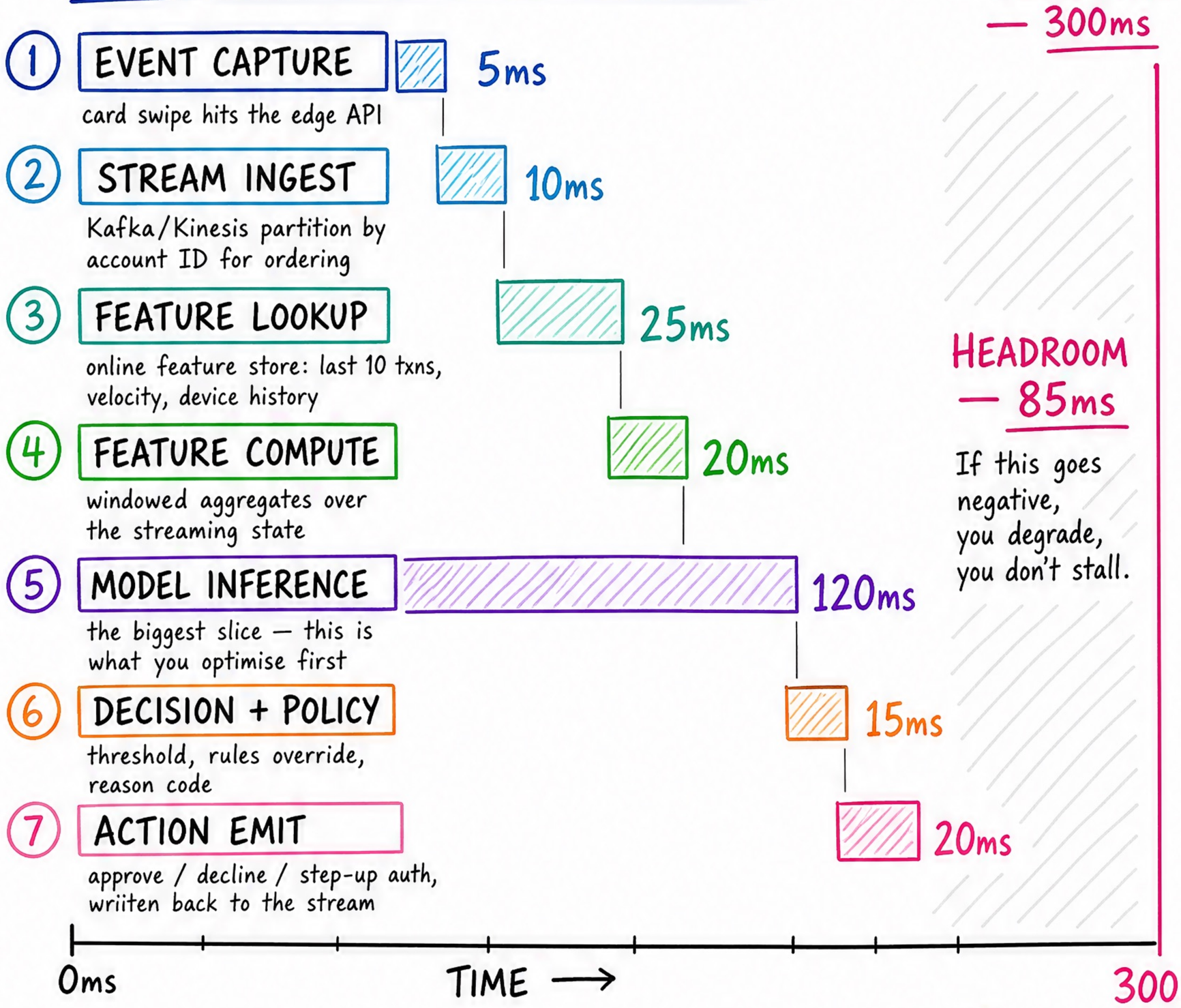
REAL-TIME AI PIPELINE

Fraud flagged during the swipe, not in tomorrow's report.

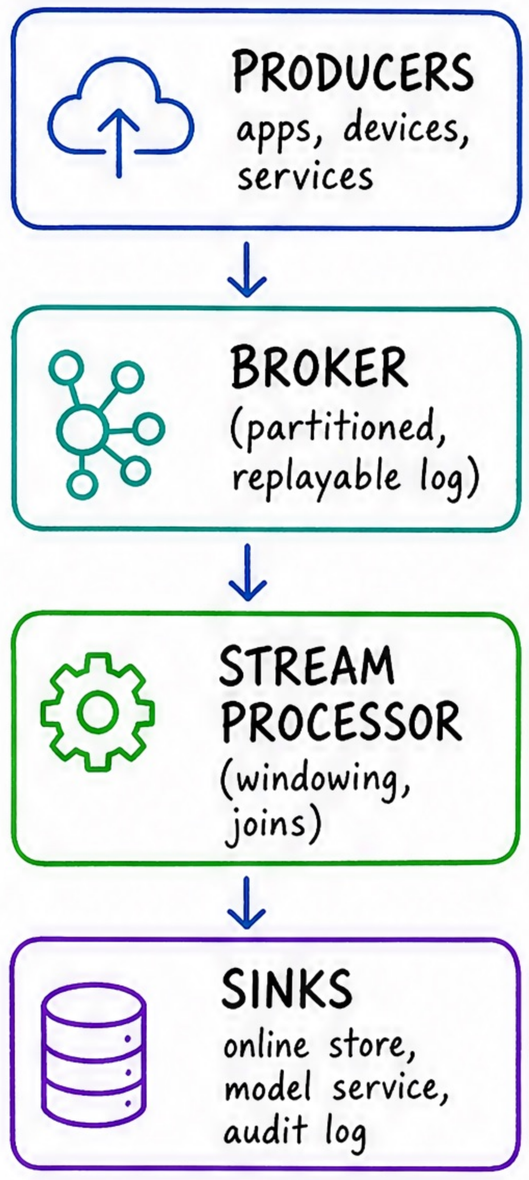
WHY THIS EXISTS

Some decisions expire. A fraud score computed five minutes after the transaction cleared is a report, not a control. A recommendation served after the user has left the page is waste. Batch infrastructure cannot meet these deadlines because it is built around throughput, not latency, and a naive request-response design collapses when events arrive faster than they can be processed. Streaming architecture exists to keep a fixed latency budget under unpredictable load, and to survive the moments when it can't.

LATENCY BUDGET WATERFALL



THE STREAM SPINE



WHEN THE BUDGET BLOWS

- **Degrade, don't queue** — serve a cached or rules-based decision rather than time out
- **Backpressure** — the processor must be able to slow producers or shed load deliberately
- **Watch the tail** — P99 latency is the user experience, the average is a comfort blanket

NEW HERE?

- **Latency budget** = the total time you're allowed, divided up in advance across stages.
- **Backpressure** = a downstream system telling upstream to slow down.
- **Windowing** = computing over the last N seconds or events instead of all history.

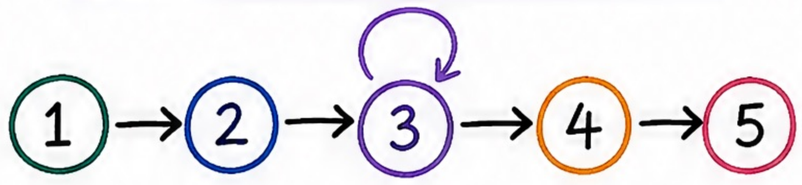
TECH FUSIONIST

Scan to Connect

BATCH AI PIPELINE

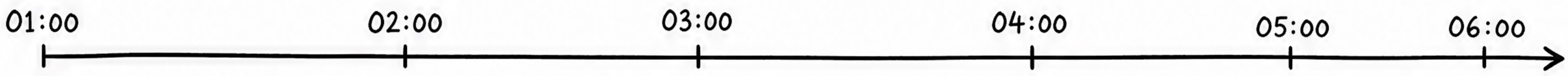
When the answer can be six hours old and cost a tenth as much.

THE ORCHESTRATOR'S VIEW



each task idempotent, retryable, independently re-runnable.

NIGHTLY SCHEDULE



01:00 — ① SNAPSHOT
freeze source tables, record watermark
20 min

01:20 — ② FEATURE BUILD
join, aggregate, backfill missing
60 min

02:20 — ③ BULK INFERENCE
score 40M records in partitioned chunks, GPU pool spun up on demand and torn down after
150 min

04:50 — ④ VALIDATE
score distribution vs yesterday, null rate, row count guardrails
20 min

if the distribution shifts beyond tolerance, **HALT** — do not publish. Yesterday's scores beat today's wrong ones.

05:10 — ⑤ PUBLISH
atomic swap into the serving table, previous version retained
15 min

WHY THIS EXISTS

Not every prediction needs to be fresh. Churn scores, propensity models, document re-embedding, nightly summarisation and catalogue enrichment all tolerate hours of staleness, and forcing them through real-time infrastructure means paying for always-on capacity and per-record overhead to serve a deadline nobody has. Batch flips the economics: bulk the work, rent the compute only while it runs, and amortise the per-call cost across millions of rows.

BATCH VS REAL-TIME — HOW TO DECIDE

- Does a decision expire in seconds? → real-time. Hours? → batch.
- Is the input a single event or the whole table? → event: real-time. Table: batch.
- Is throughput or latency the binding constraint? → throughput means batch.
- Many systems run both over the same features — that's a lambda-style split, and the features must be computed identically on both paths or you get skew.

WHAT BREAKS

- Silent partial failure — 3% of records never scored, nobody notices for a week
- Schema drift upstream — a renamed column takes the run down at 02:00
- Non-idempotent reruns — a retry double-writes and corrupts the serving table

NEW HERE?

- DAG = directed acyclic graph, the dependency map of your jobs.
- Idempotent = running it twice gives the same result as running it once.
- Watermark = the cut-off timestamp marking what this run included.

TECH FUSIONIST

Scan to Connect

HITL

HUMAN-IN-THE-LOOP ARCHITECTURE

The AI drafts the contract clause.
The lawyer still signs it.

WHY THIS EXISTS

Full automation and full manual review are both wrong answers for high-stakes work. Letting a model take irreversible actions unsupervised means a confident hallucination becomes a wire transfer, a deleted record or a legal commitment, and the failure is discovered by the customer. Reviewing everything by hand simply reinstates the bottleneck the system was built to remove. The design question is not whether humans are involved but which slice of traffic they see, how fast they can act on it, and whether their corrections ever make the system better.

AI LANE – automated

REQUEST → MODEL GENERATES DRAFT → CONFIDENCE & RISK SCORING → ROUTE

ROUTE → HIGH CONFIDENCE, LOW RISK → AUTO-EXECUTE

ROUTE → LOW CONFIDENCE OR HIGH RISK → HUMAN LANE

AUTO-EXECUTE → EXECUTE

HUMAN LANE – reviewed

REVIEW QUEUE (prioritised by risk x wait time) → REVIEWER UI (draft + retrieved evidence + model's reasoning + one-click actions) → APPROVE / EDIT / REJECT

APPROVE / EDIT / REJECT → FEEDBACK STORE

FEEDBACK STORE → REVIEW QUEUE

FEEDBACK STORE – every edit and rejection becomes evaluation data and future training signal. If you don't capture this, you've built a bottleneck, not a loop.

WHAT DECIDES THE ROUTE

- Model confidence below threshold
- Action is irreversible – money moves, a message sends, a record deletes
- Regulated domain – medical, legal, credit, hiring
- Novel input – far from anything in the evaluation set
- Random sample – always review a slice of AUTO traffic, or you go blind to drift

DESIGNING THE REVIEW ITSELF

- Surface evidence, not just the answer – a reviewer without sources is rubber-stamping
- Measure review latency – a queue nobody clears is an outage with extra steps
- Track override rate per category – it's your sharpest quality metric

NEW HERE?

Confidence score = the system's own estimate of how likely it is to be right, which is not the same as being right.

Rubber-stamping = approving without really checking, the most common way HITL quietly fails.

HITL is a seatbelt for the risky slice, not a rejection of automation.

TECH FUSIONIST

Scan to Connect

MULTI-MODEL ARCHITECTURE

One hammer, many kinds of nail — so stop using the hammer.

CAPABILITY MATRIX

no column wins every row — that's the whole argument

	Frontier LLM	Small Fast LLM	Embedding Model	Vision / OCR	Speech	Reranker
Long-form reasoning	✓	◠	✗	✗	✗	✗
Structured extraction	✓	✓	◠	✓	◠	✗
Semantic retrieval	◠	✗	✓	◠	✗	◠
Document understanding	◠	◠	◠	✓	✗	◠
Voice interface	◠	◠	✗	✗	✓	◠
Result ordering	◠	✗	◠	◠	◠	✓
Cost per 1K calls (lower is better)	▮▮▮	▮	▮	▮▮	▮▮	▮
Typical latency (lower is better)	▮▮▮	▮▮	▮▮	▮▮	▮▮	▮▮

✓ = Strong fit ◠ = Partial fit ✗ = Weak fit | ▮ = Lower / Better ▮ = Higher / Worse

WHY THIS EXISTS

Treating 'the model' as a single component is the most expensive simplification in applied AI. A frontier text model is a poor and costly OCR engine, a bad reranker and an unnecessary formatter, while a small model is excellent at exactly those narrow jobs and useless at the reasoning step.

Committing everything to one model also concentrates vendor risk: a pricing change, a deprecation or an outage hits every feature simultaneously.

Composition spreads both the cost and the risk.

HOW THEY COMPOSE IN ONE REQUEST

1. **Scanned claim PDF**
2. **Vision/OCR model** — extract text + detect stamps
3. **Embedding model** — index against policy corpus
4. **Reranker** — pick the governing clauses
5. **Frontier LLM** — reason about coverage, draft decision
6. **Small model** — format into the claims-system schema
7. **Validated JSON out**

five models, one user-visible action.
Each does the one thing it's best at.

WHAT IT COSTS YOU

- Every model is a separate dependency, SDK, rate limit and failure mode
- Formats differ — you need a normalisation layer between every hop
- Evaluation gets harder — a bad answer could have originated at any stage
- Version drift — five models deprecating on five different schedules

Modality = the kind of input a model handles — text, image, audio.

Reranker = a small model that reorders candidate results by relevance.

Multi-model is picking the right specialist per step, not running several models and hoping one is right.

TECH FUSIONIST

Scan to Connect

FAULT-TOLERANT AI ARCHITECTURE

Your provider goes down at 02:00. Your product shouldn't.

THE FAILURE MODES NOBODY PLANS FOR

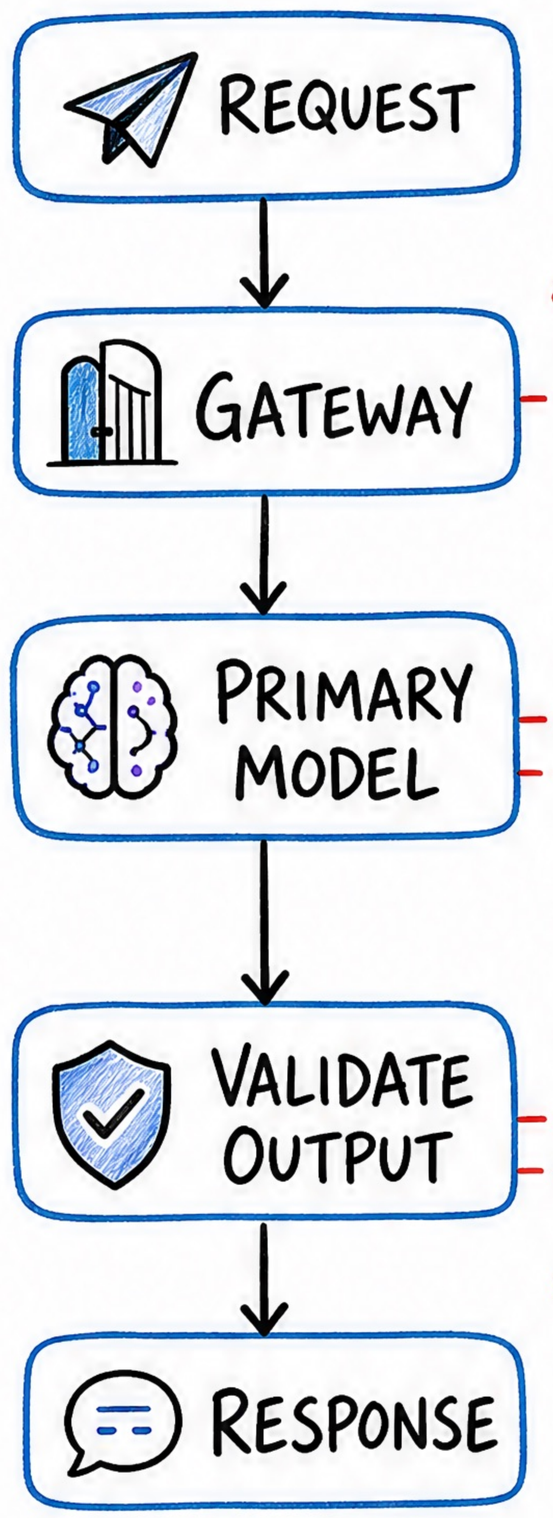
Slow success — the call returns in 40s, which is worse than an error

Confidently wrong output — no exception thrown, nothing to catch

Silent truncation — max_tokens hit mid-answer and the response looks complete

Partial agent failure — step 4 of 9 fails, leaving side effects half-applied

Provider deprecation — a model ID stops existing on a published date



CIRCUIT BREAKER OPEN — stop sending, fail fast, retry a probe every 30s. Hammering a dying service makes it worse for everyone.

RETRY with exponential backoff + jitter, capped at 2 attempts. Beyond that you're just queueing.

FAILOVER to secondary provider with an equivalent prompt. Prompts are not portable — keep a tested variant per provider.

REPAIR PASS — one constrained retry asking only for a corrected structure.

GRACEFUL DEGRADE — cached prior answer, deterministic rules-based response, or an honest 'this feature is temporarily limited'. Never a spinner that never resolves.

WHY THIS EXISTS

AI dependencies fail differently from ordinary services. They fail slowly, they fail partially, and most dangerously they fail successfully — returning a well-formed answer that happens to be wrong, which no try-catch will ever detect. On top of that you're depending on a third party whose capacity, pricing and model lineup you don't control. Fault tolerance here means assuming every call can time out, degrade, truncate or lie, and deciding in advance what the user sees when it does.

THE RULE

Every model call needs a timeout, a retry policy, a fallback and a degraded response. If any of those four is missing, you have an outage waiting on a calendar you don't own.

NEW HERE?

Circuit breaker = stop calling a failing service for a while instead of retrying it to death.
Jitter = randomising retry delays so all your clients don't retry in unison.
Graceful degradation = a worse-but-working answer instead of an error page.

TECH FUSIONIST

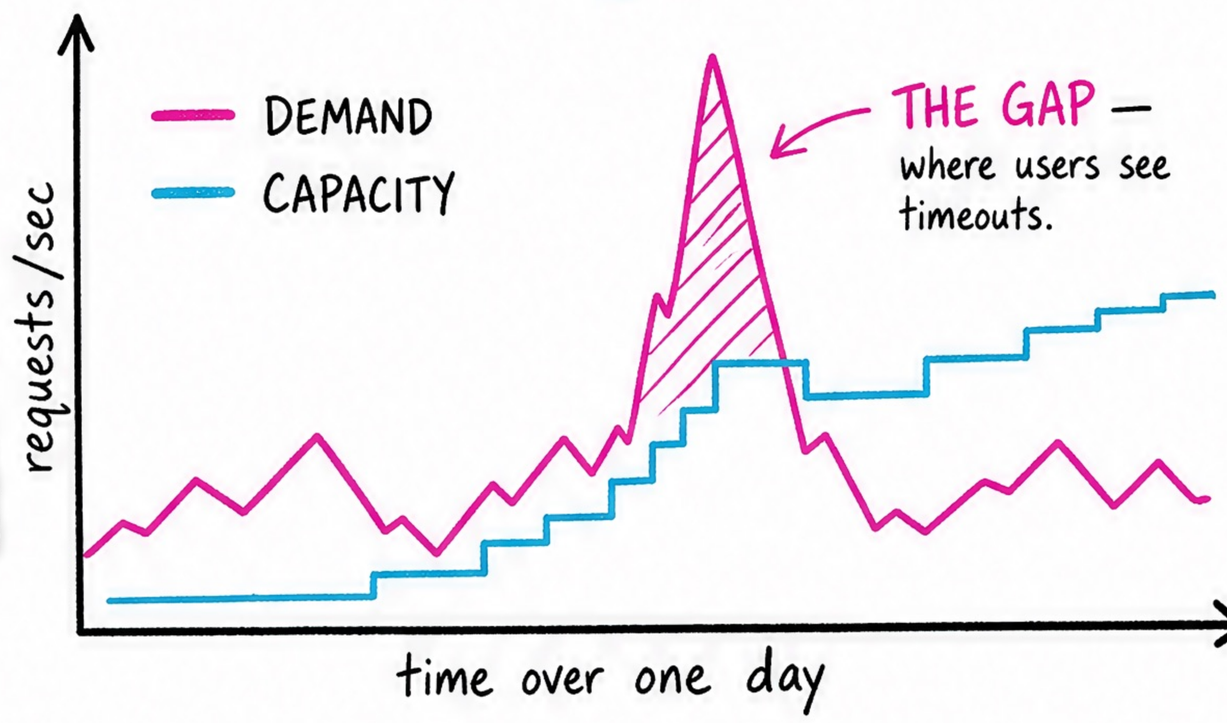
Scan to Connect

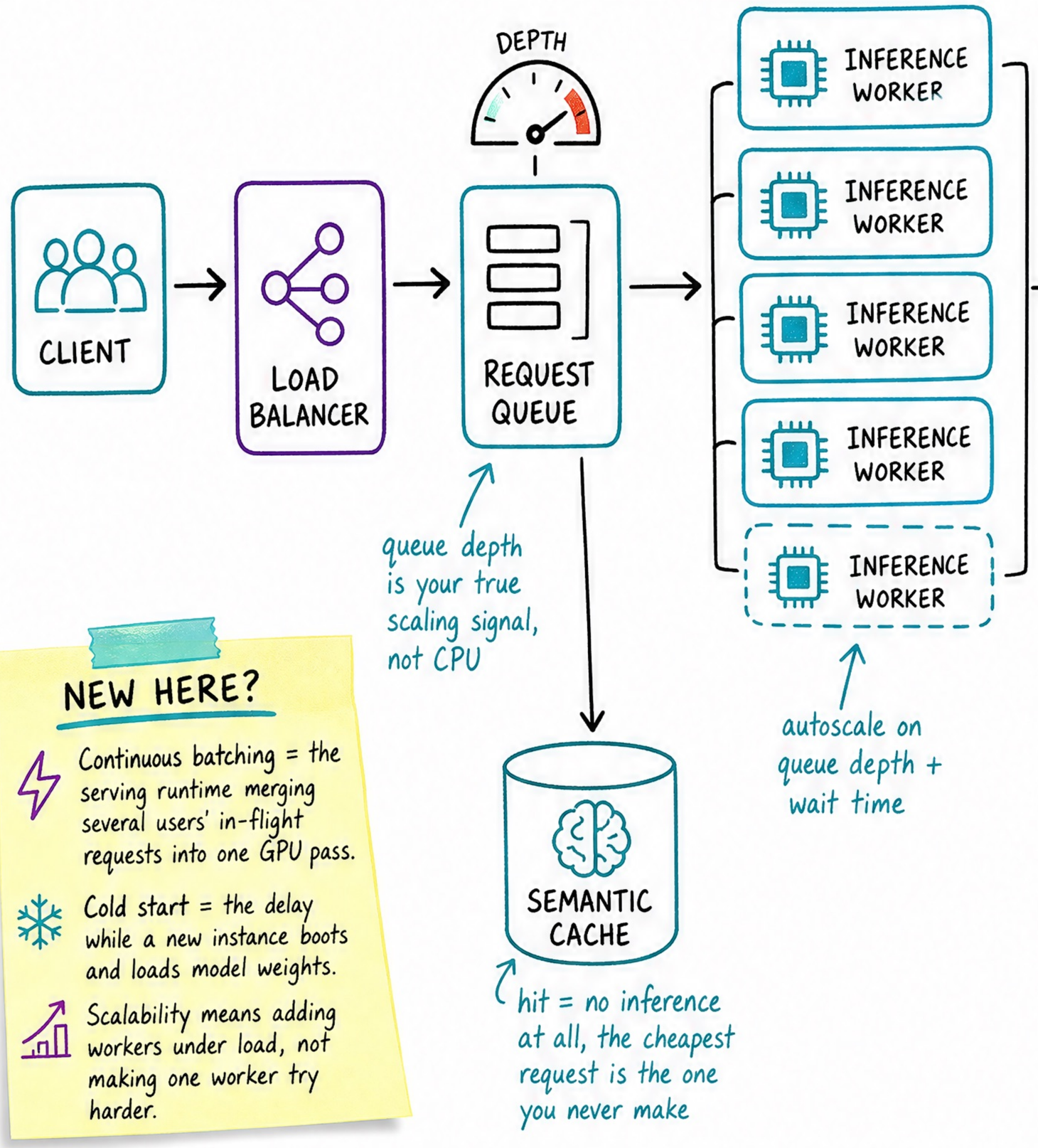
SCALABLE AI ARCHITECTURE

100 users to 100,000 overnight — and GPUs you can't just conjure in 30 seconds.

WHY THIS EXISTS

Model inference does not scale like ordinary web traffic. Each request occupies expensive accelerator memory for hundreds of milliseconds, capacity cannot be added in seconds because instances need to boot and load weights, and per-request cost stays high no matter how much traffic you have — there's no economy of scale hiding inside a token. A design that works comfortably at ten requests per second can become both unusably slow and financially unviable at a thousand, and it usually happens on the day the product finally works.





THE FOUR LEVERS, CHEAPEST FIRST

- **Cache** — exact and semantic. Free wins first.
- **Batch** — continuous batching multiplies GPU throughput several times over.
- **Route** — send the easy majority to a smaller, faster model.
- **Add capacity** — the last resort, because it's the only one that scales your bill linearly.

NEW HERE?

- Continuous batching = the serving runtime merging several users' in-flight requests into one GPU pass.
- Cold start = the delay while a new instance boots and loads model weights.
- Scalability means adding workers under load, not making one worker try harder.

TECH FUSIONIST

Scan to Connect

SECURE AI ARCHITECTURE

Treat every input as **hostile** — including the PDF a customer asked you to summarise.

PROMPT INJECTION
hidden instructions inside a doc, web page or email

SUPPLY CHAIN
poisoned documents deliberately planted in your RAG corpus

CONFUSED DEPUTY
the model queries with more privilege than the user

EXCESSIVE AGENCY
an agent with delete rights it never needed

DATA EXFILTRATION
model coaxed into echoing secrets or other tenants' context into a response or a crafted URL

1 INPUT CONTROL
sanitise untrusted content, strip instruction-like text from retrieved documents, separate system instructions from user data structurally

2 IDENTITY & AUTHZ
the model inherits the CALLER's permissions, never a service account with god rights. Retrieval filters by ACL at query time, not after.

3 TOOL SANDBOX
least-privilege tools, allow-listed domains and commands, no raw shell, no unrestricted HTTP, human approval on irreversible actions

DATA PROTECTION
PII detection and redaction before the prompt leaves your perimeter, no customer data in logs by default, retention and residency enforced

5 OUTPUT CONTROL
schema validation, content filtering, secret scanning on responses, block rendering of model-supplied links and markup

CORE: MODEL
assume it will faithfully follow any instruction it is shown, from anywhere in its context

WHY THIS EXISTS

A language model cannot reliably tell the difference between the instructions you gave it and instructions hidden inside the content it was asked to process. That single property makes every document, web page, email and API response in the context window a potential command channel, and it is not fixed by better prompting. Layer that over tools with real permissions and retrieval that reaches into sensitive corpora, and an ordinary summarisation feature becomes a way to read data the user was never entitled to see — with nothing in the logs that looks like an attack.

THE GOVERNING ASSUMPTION

Anything in the context window is untrusted input.
Anything the model can call is a privilege you granted.
Design as if both will be turned against you.

NEW HERE?

- Prompt injection = text hidden in content that hijacks the model's instructions.
- Least privilege = giving a component only the access it needs for its actual job.
- Confused deputy = a trusted component tricked into acting on behalf of someone who lacks the rights.

TECH FUSIONIST

Scan to Connect

THAT'S THE FULL STACK.

15 architectures. One system. Save this so you never lose it.

- Save this guide
- Share with your team
- Follow Tech Fusionist for more

FOLLOW

Scan to Connect to the Tech Fusionist Hub

Build Smarter. Ship Faster.

TECH FUSIONIST

Scan to Connect